ACTIVIDAD TKINTER EN GIT

Instructivo del equipo. Descárgalo y síguelo completo.

CECyTEM Plantel Huixquilucan · 1000 Firmas · Un PDF de evidencia, máximo 8 MB

Este archivo es el instructivo. No lo subas como evidencia. La evidencia es otro PDF, el de tu equipo, y se sube en SimEvaluación.

1. Qué hace el equipo

Cada equipo hace un solo programa, distinto al de los demás. El título está en la tarjeta de SimEvaluación. Hagan ese. No elijan otro.

El programa trata de lo que dice ese título. La ventana, las cajas y los botones sirven para ese tema. Además, el equipo crea una base de datos para guardar los datos que genere la aplicación.

Todo vive en un archivo: programa.py. Ahí van tres cosas:

- La cadena de conexión a esa base de datos.
- La aplicación de Tkinter que resuelve el problema del equipo.
- El CRUD: guardar, consultar, actualizar y eliminar en esa base.

Un integrante, de preferencia el líder, crea el repositorio y sube el primer programa.py. Los demás lo clonan, cada quien hace una parte, y la suben a la misma rama: main.

2. Los programas. El de tu equipo está en la tarjeta

- 1. Tablas de multiplicar del 1 al 10. Una caja, un botón Ver tabla, y del 1 al 10 el resultado. Si el número no está entre 1 y 10, un aviso.
- 2. Conversor de Celsius a Fahrenheit. Una caja de grados Celsius y un botón Convertir. F = C × 9 / 5 + 32.
- 3. Promedio de 4 calificaciones. Cuatro cajas y un botón Promedio. Muestra el promedio con un decimal.
- 4. Lista de pendientes. Una caja, un botón Agregar y una lista. Quitar borra solo la tarea seleccionada.
- 5. Contador. Una etiqueta grande en 0. Botones +1, -1 y Reiniciar.
- 6. Metros a centímetros. Una caja de metros y un botón Convertir. Muestra metros × 100.
- 7. Ficha del alumno. Cajas de nombre, grupo y número de lista. Un botón arma la ficha.
- 8. Cuadrados del 1 al 20. Un botón lista del 1 al 20 el número al cuadrado. La última línea es 400.
- 9. Propina. Caja del total y caja del porcentaje. Muestra la propina y el total a pagar.
- 10. Precio con descuento. Caja del precio y caja del porcentaje. Avisa si el descuento no está entre 0 y 100.
- 11. Adivina el número. El programa piensa un número del 1 al 20. Dice muy alto, muy bajo o le atinaste.
- 12. Total de 3 productos. Tres nombres y tres precios. Muestra cada producto y la suma.

Esos datos del programa también se guardan en la base. Por ejemplo, la tabla de multiplicar guarda el número consultado; el promedio guarda las cuatro calificaciones; la lista guarda cada pendiente. El docente ve en la tarjeta cuál les tocó.

3. La base de datos y la cadena de conexión

Creen una base nueva. No reutilicen la base 302 ni la tabla test de la otra actividad. El nombre lleva el grupo y el equipo, con guion bajo, sin espacios ni acentos. Ejemplo: tkinter_301_equipo_1.

La cadena de conexión usa los mismos datos de la clase: host localhost, usuario root y contraseña vacía. Solo cambia el nombre de la base. Va al inicio de programa.py, con el comentario de quien la escribió:

```
# Ana Lopez: cadena de conexion y base de datos
import mysql.connector
import tkinter as tk
from tkinter import messagebox

def conectar():
    return mysql.connector.connect(
        host="localhost",
        user="root",
        password="",
        database="tkinter_301_equipo_1",
    )
```

Antes de abrir la aplicación, la base y su tabla ya existen. Pueden crearlas con un script o desde la misma aplicación, pero tienen que existir. MySQL tiene que estar encendido en esa computadora.

El CRUD escribe en esa tabla, no solo en la pantalla:

- Guardar: inserta el dato que el usuario escribió.
- Consultar: lee la tabla y la muestra en la ventana.
- Actualizar: cambia un dato que ya estaba guardado.
- Eliminar: borra un registro y, al consultar de nuevo, ya no aparece.

Después de Guardar, Actualizar o Eliminar, la ventana vuelve a consultar para que se vea el cambio.

4. Cada integrante hace algo, y deja su nombre

Nadie se queda mirando. El líder reparte el trabajo. Una repartición que sí cuenta:

- Una persona: la base, la tabla y la cadena de conexión.
- Otra: la ventana de Tkinter del problema que les tocó.
- Otra: Guardar y Consultar.
- Otra: Actualizar y Eliminar.
- Si sobran personas: colores del equipo, el botón Limpiar, o el pie con los nombres de todos.

Justo arriba del cambio, en programa.py, un comentario con el nombre y lo que hizo:

```
# Ana Lopez: agrego el boton Limpiar
```

El commit lleva el nombre del cambio y el nombre de esa persona. Cambia el ejemplo por el tuyo:

```
git commit -m "Agrego el boton Limpiar - Ana Lopez"
```

Si el commit queda a nombre de otra persona, esa parte no cuenta. En una computadora compartida el nombre se cambia solo dentro de la carpeta del proyecto, sin --global, para no borrar el usuario de la siguiente clase.

5. Qué significa cada palabra

- Guardar: el archivo queda en la carpeta de esta computadora. En el editor es Ctrl+S, o Archivo y luego Guardar.
- git add: preparas programa.py para el siguiente registro.
- Commit, o comprometer: el cambio queda registrado, con un mensaje y con tu nombre.
- Push, o subir: el cambio sale de esta computadora y llega a GitHub, a la rama main.
- Pull, o bajar: traes a tu carpeta lo que tus compañeros ya subieron.
- Clonar: bajas el proyecto completo la primera vez.
- Rama main: la única rama del equipo. No creen otra.
- Repositorio: el proyecto en GitHub. El nombre lo dice la tarjeta. Ejemplo: tkinter-301-equipo-1.

6. Instalar Git en la computadora

1. Abre la carpeta donde vas a trabajar. Haz clic en la barra de la dirección, escribe powershell y pulsa Enter.
2. Escribe git --version y pulsa Enter.
3. Si aparece un número, por ejemplo git version 2.45.0, Git ya está.
4. Si dice que git no se reconoce, entra a https://git-scm.com/download/win y baja Git para Windows.
5. En el instalador pulsa Next hasta el final. En la pantalla de PATH deja la opción Git from the command line and also from 3rd-party software.
6. Cierra esa ventana negra y ábrela otra vez. Vuelve a probar git --version.

```
git --version
```

7. Revisa que Git esté a tu nombre

Hazlo en cada computadora, antes de comprometer. Si la máquina la usó otra persona, el commit sale con el nombre de ella.

```
git config --global user.name
git config --global user.email
```

Tiene que salir tu nombre y el correo de tu cuenta de GitHub. El correo se ve en GitHub: tu foto, Settings, Emails.

Si no es el tuyo, entra primero a la carpeta del proyecto y cambia el usuario solo ahí. Estas líneas no llevan --global:

```
git config user.name "Ana Lopez"
git config user.email "ana.lopez@gmail.com"
git config user.name
git config user.email
```

Vuelve a preguntar. Tiene que salir tu nombre. Si sigue saliendo otra persona, no estás dentro de la carpeta del proyecto.

8. El líder crea el proyecto y sube el primer programa

1. Entra a github.com con tu cuenta. Junto a tu foto pulsa + y elige New repository.
2. En Repository name escribe el nombre exacto de la tarjeta. Márcalo Public.
3. Deja sin marcar Add a README. Si lo marcas, el primer envío se atasca. Pulsa Create repository y deja esa página abierta.
4. En Settings, Collaborators (o Access y luego Collaborators), pulsa Add people. Invita a cada integrante y al docente. Cada quien acepta en la campana.
5. En la ventana negra, crea la carpeta en Documentos y entra:

```
cd $env:USERPROFILE\Documents
mkdir tkinter-301-equipo-1
cd tkinter-301-equipo-1
```

Cambia tkinter-301-equipo-1 por el nombre de la tarjeta.

6. Escribe el programa en Python y guárdalo con Archivo, Guardar como. El archivo se llama exactamente programa.py y queda dentro de esa carpeta, no en el escritorio.
7. Si la computadora no está a tu nombre, cambia el usuario con el paso 7, ya dentro de esta carpeta.
8. Sube el primer archivo. Los comandos van uno por uno. En remote add cambia TU-USUARIO por tu usuario de GitHub:

```
git init
git branch -M main
git add programa.py
git status
git commit -m "Subo el programa inicial - Ana Lopez"
git remote add origin https://github.com/TU-USUARIO/tkinter-301-equipo-1.git
git push -u origin main
```

En el push se abre el navegador. Entra con tu cuenta de GitHub. Si la ventana negra pide Password, no escribas la contraseña de GitHub. Ciérrala y deja que se abra el navegador. GitHub ya no acepta esa contraseña en la terminal.

Recarga la página del repositorio. Tiene que verse programa.py. Pulsa el botón verde Code, elige HTTPS, copia la dirección y pégala en el chat del equipo.

9. Los demás clonan

1. Primero acepta la invitación. Sin eso, el clone pide permiso y no entra.
2. Abre PowerShell y clona la dirección que copió el líder. Luego entra a la carpeta:

```
cd $env:USERPROFILE\Documents
git clone https://github.com/USUARIO-DEL-LIDER/tkinter-301-equipo-1.git
cd tkinter-301-equipo-1
```

3. Pregunta el usuario. Si no es el tuyo, cámbialo dentro de esta carpeta, sin --global.
4. Abre programa.py desde esa carpeta. Tiene que ser el programa del equipo, no uno nuevo.

10. Cada vez que hagas un cambio

Este bloque se repite. No es solo la primera vez. Cada persona, cada cambio, en este orden. Entra una sola persona a la vez.

1. Guardar. En el editor pulsa Ctrl+S. Si no guardas, Git sube la versión vieja. El nombre del archivo sigue siendo programa.py.
2. Abre PowerShell dentro de la carpeta. La ruta termina en el nombre del repositorio. Si no, escribe cd y el nombre de la carpeta.
3. Revisa tu nombre con git config user.name. Si sale otra persona, cámbialo antes de seguir.
4. Bajar lo último, para no pisar el trabajo de otro:

```
git pull origin main
```

5. Abre el programa.py de esa carpeta. Haz solo tu parte. Arriba del cambio deja el comentario con tu nombre. Guarda otra vez con Ctrl+S.
6. Preparar el archivo y revisar que Git sí lo ve en rojo o en verde:

```
git add programa.py
git status
```

7. Comprometer. El mensaje es el nombre del cambio y tu nombre:

```
git commit -m "Agrego el boton Limpiar - Ana Lopez"
```

8. Subir a la rama main:

```
git push origin main
```

9. Comprobar. Este comando debe decir working tree clean. Luego recarga GitHub: tu usuario aparece junto

al cambio.

```
git status
```

Orden corto, para no saltarse ninguno: guardar, bajar, preparar, comprometer, subir, comprobar.

11. Si algo sale mal

Si pasa esto: Escribe git y dice que no se reconoce como comando.
Haz esto: Git no está instalado, o la ventana negra se abrió antes de instalarlo. Instálalo desde git-scm.com/download/win, deja la opción de PATH, cierra PowerShell y ábrelo de nuevo.

Si pasa esto: El commit en GitHub está a nombre de otra persona.
Haz esto: Estás en una computadora compartida. Entra a la carpeta del proyecto y pon tu nombre sin --global. El commit viejo se queda con el otro nombre. El siguiente tiene que salir con el tuyo. No intentes reescribir los commits ya subidos.

Si pasa esto: La ventana negra pide Password o Username.
Haz esto: No escribas la contraseña de GitHub. Ciérrala. Deja que se abra el navegador e inicia sesión ahí. GitHub ya no acepta la contraseña de la cuenta dentro de la terminal.

Si pasa esto: Dice rejected, failed to push o fetch first.
Haz esto: Alguien subió antes que tú. Baja sus cambios y vuelve a subir. Si se abre un editor raro, pulsa Esc, escribe :wq y pulsa Enter. Luego repite el push.

```
git pull origin main --no-edit
git push origin main
```

Si pasa esto: git status dice nothing to commit, working tree clean, pero tu cambio no está.
Haz esto: No guardaste, o editaste una copia del escritorio. Abre el programa.py que está dentro de la carpeta del repositorio, guarda con Ctrl+S y vuelve a hacer add, commit y push.

Si pasa esto: Dice fatal: not a git repository.
Haz esto: La ventana negra no está dentro de la carpeta del proyecto. Escribe cd $env:USERPROFILE\Documents y luego cd y el nombre del repositorio.

Si pasa esto: Dice remote origin already exists.
Haz esto: La dirección ya estaba puesta. No la agregues otra vez. Para verla: git remote -v. Si apunta a otro usuario, corrígela:

```
git remote set-url origin https://github.com/TU-USUARIO/tkinter-301-equipo-1.git
```

Si pasa esto: Dice 403, Permission denied o Authentication failed.
Haz esto: No has aceptado la invitación, o entraste con otra cuenta. Abre GitHub, la campana, y pulsa Accept invitation. El líder te agrega en Settings, Collaborators. Luego cierra sesión del navegador y entra con tu cuenta.

Si pasa esto: En programa.py aparecen líneas con <<<<<<<, ======= o >>>>>>>.
Haz esto: Dos personas editaron las mismas líneas. No borres el archivo. Deja el código de los dos, quita esas tres marcas, guarda, y comprometé: Junto los dos cambios - Tu Nombre. A partir de ahí, una sola persona sube a la vez.

Si pasa esto: Se abre una pantalla donde las letras no se escriben normal, o dice vim.
Haz esto: Caíste en el editor de Git. Pulsa Esc. Escribe :wq y pulsa Enter. Si no sale, cierra esa ventana y abre PowerShell otra vez dentro de la carpeta.

Si pasa esto: En GitHub el archivo se llama Programa.py, o no aparece.
Haz esto: El nombre exacto es programa.py, todo en minúsculas, dentro de la carpeta del repositorio. Si lo guardaste en el escritorio, Git no lo ve.

Si pasa esto: El push dice que la rama no es main, o estás en master.
Haz esto: El equipo solo usa main. Dentro de la carpeta:

```
git branch -M main
git push -u origin main
```

Si pasa esto: La aplicación no abre, o dice que no encuentra mysql.connector.
Haz esto: Se prueba en la computadora, con Python. En esa computadora: pip install mysql-connector-python. MySQL tiene que estar encendido. El celular no abre una ventana de Tkinter.

Si pasa esto: Dice que no puede conectar, o que la base no existe.
Haz esto: Revisa la cadena: host localhost, user root, password vacío, y el database con el nombre que sí crearon. Creen la base antes de pulsar Guardar. No usen la base 302 de la otra actividad.

Si pasa esto: Aparece un aviso de LF will be replaced by CRLF.
Haz esto: No es un error. Puedes seguir. No cambia el programa.

Si pasa esto: Dos personas empujan al mismo tiempo y uno pierde su botón.
Haz esto: Entren de uno en uno. Quien va a editar, primero hace git pull. Quien ya subió, avisa en el chat antes de que el siguiente abra el archivo.

12. Si estás en el celular

El programa de Tkinter y la base de datos se prueban en una computadora. El celular no abre esa ventana. El celular sirve para leer el proyecto, aceptar la invitación, editar programa.py, comprometer y subir, cuando no tienes la computadora a la mano.

No hace falta pagar. En los dos teléfonos el camino seguro es el navegador, en https://github.com. La aplicación GitHub, la oficial, sirve para aceptar la invitación y ver el repositorio. El cambio del archivo se hace más claro en el navegador.

En el celular, el botón Commit changes guarda, compromete y sube en un solo paso. No busques tres botones distintos.

Android

1. Instala la aplicación GitHub, la oficial, de GitHub, Inc., desde Play Store. Es gratis. Ábrela, entra con tu cuenta y acepta la invitación del equipo.
2. Para editar, abre Chrome y entra a https://github.com con la misma cuenta.
3. Abre el repositorio del equipo y entra al archivo programa.py.
4. Pulsa el ícono del lápiz. Si no lo ves, pulsa los tres puntos y elige Edit.
5. Haz tu parte. Arriba del cambio escribe el comentario, por ejemplo: # Ana Lopez: agrego el boton Limpiar.
6. Baja hasta Commit changes. En el recuadro del mensaje escribe el cambio y tu nombre: Agrego el boton Limpiar - Ana Lopez.
7. Deja la rama main. No crees otra. Pulsa Commit changes.
8. Recarga el archivo. Tu nombre tiene que verse en el último cambio.

Si Chrome abre la versión recortada y no aparece el lápiz, pulsa el menú del navegador y elige Sitio de escritorio.

Los comandos git de la computadora no son obligatorios en el teléfono. Si aun así los quieres, la aplicación es Termux y se instala desde F-Droid, no desde la versión vieja de Play Store. Dentro se escribe pkg install git y luego los mismos comandos. Para esta actividad basta el navegador.

iPhone

1. Instala la aplicación GitHub, la oficial, desde la App Store. Es gratis. Entra con tu cuenta y acepta la

invitación.

2. Para editar, abre Safari y entra a https://github.com con la misma cuenta.
3. Si la página se ve apretada, pulsa el botón aA de la barra y elige Sitio de escritorio.
4. Abre el repositorio, entra a programa.py y pulsa el lápiz.
5. Haz tu parte y deja el comentario con tu nombre arriba del cambio.
6. Baja a Commit changes. El mensaje lleva el cambio y tu nombre. La rama se queda en main. Pulsa Commit changes.
7. Recarga y revisa que tu usuario aparezca en ese cambio.

Existe también Working Copy, en la App Store, por si quieren clonar como en la computadora. Si esa aplicación pide una compra para subir a GitHub, no la paguen. Safari en github.com sube el cambio gratis.

No entreguen el programa desde una aplicación de editor de Python del celular. Esa aplicación no lo sube a GitHub y el docente no la ve.

13. La evidencia que sí se sube a SimEvaluación

Es un solo PDF por equipo, de 8 MB o menos. Lo sube un integrante, de preferencia el líder. Si lo vuelven a subir, reemplaza el anterior. Este instructivo no cuenta como esa evidencia.

- Portada: de qué trata el programa, el nombre del equipo, los integrantes y el enlace del repositorio.
- La base de datos creada y la tabla con datos.
- La cadena de conexión dentro de programa.py.
- La ventana de Tkinter funcionando: guardar, consultar, actualizar y eliminar.
- El comentario de cada integrante dentro del código, con su nombre.

Antes de elegir el archivo, en SimEvaluación se marcan las casillas. Sin eso, el botón de subir no se activa.

14. Orden final, para no perderse

1. Vean en la tarjeta cuál programa les tocó y cómo se llama el repositorio.
2. El líder crea la base, el programa.py con la conexión, el Tkinter y el CRUD, y lo sube.
3. Cada compañero clona, pone su usuario, guarda su cambio, lo compromete con su nombre y lo sube a main.
4. Si sale un error, busquen la frase en la sección 11 y hagan solo ese remedio.
5. Si están en el celular, usen la app GitHub para la invitación y el navegador para el commit.
6. Armen el PDF de evidencia y súbanlo en SimEvaluación. No suban este instructivo.